

React Native 学習まとめ

家計簿アプリ開発を通じて学んだ React Native の技術をまとめた資料です。
コードの書き方だけでなく「なぜそう書くのか」もわかるようにまとめています。

1. コンポーネント・Props・State

React Native とは

スマホアプリ（iOS・Android）を JavaScript / TypeScript で作れるフレームワークです。
画面の部品を「コンポーネント」という単位で作り、組み合わせて画面を構成します。

コンポーネントとは

画面のパーツを部品として切り出したものです。ボタン・テキスト・入力欄など、画面を構成するすべてのものがコンポーネントです。

レゴブロックのように、小さな部品を組み合わせで大きな画面を作るイメージです。

```
// 最もシンプルなコンポーネント
export default function MyComponent() {
  return <Text> こんにちは </Text>
}
```

コンポーネントは**必ず1つの要素を return** しなければなりません。複数の要素を返したいときは
<View> で囲みます。

```
// 複数の要素は View で囲む
export default function MyComponent() {
  return (
    <View >
      <Text > タイトル </Text >
      <Text > 本文 </Text >
    </View >
  )
}
```

Props とは

親コンポーネントから子コンポーネントに値を渡す仕組みです。

関数の引数のようなイメージで、外から値を受け取ってコンポーネントの表示を変えられます。

```
// 親コンポーネント：name という値を渡す
<MyComponent name="太郎" />

// 子コンポーネント：name を受け取って使う
type Props = {
  name: string; // TypeScript で受け取る型を定義する
}
export default function MyComponent({ name }: Props) {
  return <Text > {name}さん、こんにちは </Text >
  // → 「太郎さん、こんにちは」と表示される
}
```

ポイント： { name }: Props の {} は分解代入です。Props オブジェクトの中から name だけを取り出しています。

State とは

コンポーネントが持つ「変化する値」です。

State が変わると React Native が自動で画面を再描画してくれます。

普通の変数（`let count = 0`）を変えても画面は更新されません。**必ず `useState` を使う**必要があります。

```
// useState の書き方
const [count, setCount] = useState(0);
//      ↑値      ↑更新関数  ↑初期値

// 値を読むとき
<Text> {count} </Text>

// 値を更新するとき（必ず setter 関数を使う）
setCount(count + 1);
```

なぜ setter 関数を使うのか： `count = count + 1` と直接書いても画面は更新されません。 `setCount()` を使うことで React Native に「値が変わったから再描画して」と伝えられます。

2. よく使うコンポーネント

View（レイアウトの箱）

画面のレイアウトを作るための透明な箱です。HTML の `<div>` に相当します。

```
<View style={{ padding: 16 }}>
  <Text> 中身 </Text>
</View>
```

Text（テキスト表示）

文字を表示するコンポーネントです。React Native では `<Text>` 以外の場所に文字を書くとエラーになります。

```
<Text style={{ fontSize: 24, fontWeight: 'bold' }}> タイトル </Text>
```

TextInput（テキスト入力）

ユーザーが文字を入力できるコンポーネントです。 `value` と `onChangeText` をセットで使います。

```
const [text, setText] = useState('');
```

```
<TextInput placeholder="入力してください" // 何も入力していないときに薄く表示されるテキスト value={text} // 今の値
```

ポイント： `value={text}` と `onChangeText` を両方書かないと、入力した文字が画面に反映されません。これを「制御されたコンポーネント」といいます。

FlatList（リスト表示）

配列のデータをリスト形式で表示するコンポーネントです。

`ScrollView` より効率的な理由は、画面外のアイテムを描画しないためです（大量データに強い）。

```
<FlatList data={items} // 表示したい配列 keyExtractor={({item}) => item.id} // 各アイテムを区別するユニークなキ
```

Button・TouchableOpacity（ボタン）

```
// シンプルなボタン（デザインのカスタマイズが難しい）
```

```
<Button title="送信" onPress={() => console.log('押した')} />
```

```
// デザインをカスタマイズしたいときは TouchableOpacity
```

```
<TouchableOpacity onPress={() => console.log('押した')} style={styles.button}>
```

```
<Text> カスタムボタン </Text>
```

```
</TouchableOpacity>
```

つまずきポイント： `onPress` に関数を渡すときの書き方

```
// 誤り：画面が表示された瞬間に実行されてしまう（即時実行）
```

```
onPress={fn()}
```

```
// 正しい：ボタンを押したときだけ実行される
```

```
onPress={() => fn()}
```

```
onPress={fn} // 引数が不要なときはこの書き方でもOK
```

`fn()` の `()` は「今すぐ実行する」という意味です。`onPress` には「押されたときに呼ぶ関数」を渡す必要があるなので、`()` をつけてはいけません。

3. StyleSheet ・ Flexbox

StyleSheet とは

コンポーネントにデザイン（見た目）を適用する仕組みです。CSS に似ていますが、React Native 独自のプロパティ名があります（例： `backgroundColor`、 `fontSize` など）。

```
// StyleSheet.create() でスタイルを定義する
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: 'white',
    padding: 16,
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    textAlign: 'center',
    color: '#333',
  },
})

// style プロパティで適用する
<View style={styles.container}>
  <Text style={styles.title}> タイトル </Text >
</View >
```

Flexbox（レイアウト）

画面の中で要素をどう並べるかを決める仕組みです。React Native のデフォルトは**縦方向（column）**です（Webは横方向が多いので注意）。

```
// 横並びにする
<View style={{ flexDirection: 'row' }}>
  <Text > 左 </Text >
  <Text > 右 </Text >
</View >

// 中央揃えにする
<View style={{ justifyContent: 'center', alignItems: 'center' }}>
  <Text > 中央 </Text >
</View >

// 残りのスペースをすべて埋める
<View style={{ flex: 1 }}>
  <Text > このViewが残りのスペースを占める </Text >
</View >
```

プロパティ	役割
flexDirection: 'row'	子要素を横並びにする
justifyContent: 'center'	主軸（デフォルトは縦）の中央に配置
alignItems: 'center'	交差軸（デフォルトは横）の中央に配置
flex: 1	残りのスペースをすべて占有する

4. 画面遷移（Expo Router）

Expo Router とは

ファイルのパスがそのままURLになる画面遷移の仕組みです。
app/ フォルダの中にファイルを置くだけで自動的にルートが作成されます。

```
app/
├─ index.tsx      →  /           (トップページ)
├─ login.tsx      →  /login      (ログイン画面)
├─ signup.tsx     →  /signup     (サインアップ画面)
├─ settings.tsx   →  /settings   (設定画面)
└─ (tabs)/
    └─ kakeibo.tsx →  /(tabs)/kakeibo (家計簿画面)
```

注意： ファイル名とルートパスは一致します。 `settings.tsx` のルートは `/settings` です。 `/setting` (sなし) と書くと画面が見つかりません。

router の使い方

```
import { router } from 'expo-router';
```

```
// 画面を積み重ねる（戻るボタンで前の画面に戻れる）
```

```
router.push('/settings');
```

```
// 今の画面を置き換える（戻るボタンで前の画面に戻れない）
```

```
router.replace('/login');
```

使い分けの目安：

- ログイン成功後 → `replace` （ログイン画面に戻らせたくないため）
- 通常の画面移動 → `push` （戻れた方が自然なため）

Stack.Screen（画面の設定）

`_layout.tsx` で各画面のヘッダーなどを設定します。

```
<Stack >
  <Stack.Screen name="(tabs)" options={{ headerShown: false }} />
  <Stack.Screen name="login" options={{ headerShown: false }} />
  <Stack.Screen name="settings" options={{ title: '各種設定' }} />
</Stack >
```

5. AsyncStorage（ローカル保存）

AsyncStorage とは

スマホのストレージ（内部記憶）にデータを保存する仕組みです。アプリを閉じててもデータが残ります。

ただし**文字列しか保存できない**ため、配列やオブジェクトを保存するときは `JSON.stringify()` で文字列に変換し、取り出すときは `JSON.parse()` で元に戻す必要があります。

```
import AsyncStorage from '@react-native-async-storage/async-storage';

// 保存：オブジェクト → JSON文字列に変換してから保存
const data = [{ id: '1', name: '食費', amount: 500 }];
await AsyncStorage.setItem('expenses', JSON.stringify(data));

// 取得：JSON文字列 → オブジェクトに変換してから使う
const raw = await AsyncStorage.getItem('expenses');
const data = JSON.parse(raw ?? '[]');
// ?? '[]' は「rawがnullなら空配列を使う」という意味

// 削除
await AsyncStorage.removeItem('expenses');
```

現在の家計簿アプリでは Supabase（クラウド）を使っているため AsyncStorage は不使用ですが、ローカル保存の概念として重要です。

6. CRUD 操作

CRUD とは

アプリで必ず必要になる4つの基本操作の略です。

頭文字	英語	日本語	例
C	Create	作成	支出を追加する
R	Read	読取	支出一覧を表示する

頭文字	英語	日本語	例
U	Update	更新	支出の金額を修正する
D	Delete	削除	支出を削除する

State での CRUD

```
// Create（追加）：スプレッド構文で既存の配列に新しいアイテムを追加
setItems([...items, newItem]);
// [...items] は「items の中身をすべて展開する」という意味
// 結果：[既存アイテム1, 既存アイテム2, ..., 新しいアイテム]

// Delete（削除）：filter で条件に一致しないアイテムだけ残す
setItems(items.filter((item) => item.id !== targetId));
// item.id が targetId と違うものだけ残す = targetId のものを除く

// Update（更新）：map で対象アイテムだけ中身を変える
setItems(items.map((item) =>
  item.id === targetId
    ? { ...item, name: '新しい名前' } // 対象アイテム：nameだけ変える
    : item                             // それ以外：そのまま
));
```

7. カテゴリ別集計

reduce（配列を1つの値にまとめる）

reduce は配列のすべての要素を使って1つの値を計算するメソッドです。

```
// 合計金額を計算する例
const total = items.reduce((acc, item) => acc + item.amount, 0);
//                                     ↑合計値   ↑今のアイテム           ↑初期値

// 処理の流れ (items = [{amount:500}, {amount:300}, {amount:200}] の場合)
// 1回目: acc=0,   item.amount=500   → 0+500   = 500
// 2回目: acc=500, item.amount=300   → 500+300 = 800
// 3回目: acc=800, item.amount=200   → 800+200 = 1000
// 結果: 1000
```

カテゴリ別に合計を集計する

```
// { 食費: 3000, 交際費: 1500 } のようなオブジェクトを作る
const categoryTotals = items.reduce((acc, item) => {
  // acc[item.category] がまだない (undefined) なら 0 として扱う
  acc[item.category] = (acc[item.category] ?? 0) + item.amount;
  return acc;
}, {} as Record<string, number>);
// {} as Record<string, number> は「キーが文字列、値が数値のオブジェクト」という型定義
```

Object.entries (オブジェクトをループする)

```
// { 食費: 3000, 交際費: 1500 } を [['食費', 3000], ['交際費', 1500]] に変換してループ
Object.entries(categoryTotals).forEach(([category, total]) => {
  console.log(`${category}: ${total}円`);
  // → 「食費: 3000円」「交際費: 1500円」と出力される
});
```

8. バリデーション

バリデーションとは

ユーザーの入力値が正しいかどうかを確認する処理です。
 おかしい値のままデータベースに保存されることを防ぎます。

```

const handleSubmit = () => {
  // 未入力チェック
  if (inputText === '' || inputNumber === '') {
    setErrorMessage('品名と金額を入力してください');
    return; // return で以降の処理を止める
  }

  // 数値チェック
  if (Number(inputNumber) <= 0) {
    setErrorMessage('金額は1円以上を入力してください');
    return;
  }

  // ここまで来たら入力値は正常
  setErrorMessage('');
  // 保存処理へ...
}

```

ポイント： `return;` を書くことで「ここで処理を終了する」という意味になります。バリデーションに引っかかった場合、以降の保存処理が実行されなくなります。

9. 日付管理

Date オブジェクト

JavaScript の組み込みオブジェクトで、日付・時刻を扱います。

```

const d = new Date();           // 現在の日時を取得
d.getFullYear()                 // 年（例：2026）
d.getMonth() + 1                // 月（0～11 の値なので +1 が必要！）
d.getDate()                     // 日（例：10）
d.getHours()                    // 時（例：15）
d.getMinutes()                  // 分（例：30）

```

つまずきポイント： `getMonth()` は 0 始まり

`getMonth()` は 1月 → 0、2月 → 1、...、12月 → 11 を返します。
表示用には必ず +1 してください。

padStart（ゼロ埋め）

05 のように1桁の数字を2桁に揃えるメソッドです。

padStart は**文字列メソッド**なので、数値の場合は先に String() で文字列に変換する必要があります。

```
String(5).padStart(2, '0')    // → "05" (2桁になるまで左に '0' を埋める)
String(12).padStart(2, '0')   // → "12" (すでに2桁なのでそのまま)

// 日付文字列を作る例
const dateStr = `${d.getFullYear()}-${String(d.getMonth()+1).padStart(2, '0')}-${String(
// → "2026-05-10"
```

月ごとのフィルタリング

```
// "2026-05" で始まるデータだけを取り出す
const filteredItems = items.filter((item) =>
  item.date?.slice(0, 7) === currentMonth
  // "2026-05-10".slice(0, 7) → "2026-05"
);
```

オプションルチェーン（?.）

値が null や undefined のときでもエラーにならない書き方です。

```
// date が null の場合
item.date.slice(0, 7)    // → エラー (null に slice はない)
item.date?.slice(0, 7)   // → undefined (エラーにならない)
```

10. グラフ表示（react-native-chart-kit）

react-native-chart-kit とは

React Native でグラフを表示するためのライブラリです。棒グラフ・折れ線グラフ・円グラフなどが使えます。

```
npx expo install react-native-chart-kit react-native-svg
```

PieChart (円グラフ)

カテゴリ別の割合を円グラフで表示します。

```
import { PieChart } from 'react-native-chart-kit';
import { Dimensions } from 'react-native';
```

```
// 画面の幅を取得（グラフのサイズに使う）
```

```
const screenWidth = Dimensions.get('window').width;
```

```
// グラフのデータ形式
```

```
const data = [
  {
    name: '食費',
    population: 3000,           // グラフの値
    color: '#FF6384',          // グラフの色
    legendFontColor: '#333',    // 凡例のテキスト色
  },
  {
    name: '交際費',
    population: 1500,
    color: '#36A2EB',
    legendFontColor: '#333',
  },
];
```

```
<PieChart data={data} width={screenWidth} height={200} chartConfig={{ color: (opacity = 1) => `rgba(0,
```

%（モジュロ演算子）でカラーを自動割り当て

カテゴリが増えても色が足りなくならないように、モジュロ演算子で色を循環させます。

```
const COLORS = ['#FF6384', '#36A2EB', '#FFCE56'];

// index % COLORS.length は「index を COLORS.length で割った余り」
// index=0 → 0%3=0 → '#FF6384'
// index=1 → 1%3=1 → '#36A2EB'
// index=2 → 2%3=2 → '#FFCE56'
// index=3 → 3%3=0 → '#FF6384' (また最初に戻る)
const color = COLORS[index % COLORS.length];
```

11. Supabase (バックエンド・データベース)

Supabase とは

データベース・認証・ストレージなどのバックエンド機能をまとめて提供するサービスです。自分でサーバーを用意しなくても、クラウド上にデータを保存・取得できます。

セットアップ

```
// lib/supabase.ts
import { createClient } from '@supabase/supabase-js';

export const supabase = createClient(
  process.env.EXPO_PUBLIC_SUPABASE_URL!, // SupabaseプロジェクトのURL
  process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY! // 公開用APIキー
);
// ! は「この値は必ず存在する」とTypeScriptに伝えるマーク
```

CRUD

```
// Create (追加)
await supabase.from('expenses').insert({
  name: '食費',
  amount: 500,
  category: '食費',
  date: '2026-05-10'
});

// Read (取得)
const { data } = await supabase.from('expenses').select('*');
// select('*') は「すべての列を取得する」という意味

// Delete (削除)
await supabase.from('expenses').delete().eq('id', targetId);
// .eq('id', targetId) は「id が targetId と等しい行」を指定
```

エラーハンドリング

Supabase の関数はすべて { data, error } の2つを返します。
必ずエラーチェックをするようにしましょう。

```
const { data, error } = await supabase.from('expenses').insert({ ... });

if (error) {
  // エラーがある場合
  Alert.alert('エラー', error.message);
  return;
}

// ここまで来たら成功
console.log('保存成功', data);
```

12. 認証 (Supabase Auth)

認証とは

「あなたは誰ですか？」を確認する仕組みです。
メールアドレスとパスワードを使ってユーザーを識別します。

サインアップ (アカウント作成)

```
const { data, error } = await supabase.auth.signUp({
  email: email,
  password: password,
});

if (error) {
  Alert.alert('エラー', error.message);
  return;
}

// 成功 : ホーム画面へ移動
router.replace('/(tabs)/kakeibo');
```

ログイン

```
const { data, error } = await supabase.auth.signInWithPassword({
  email: email,
  password: password,
});

if (error) {
  Alert.alert('エラー', error.message);
  return;
}

router.replace('/(tabs)/kakeibo');
```


ログアウト

```
await supabase.auth.signOut();
router.replace('/login');
```

13. ルート保護

ルート保護とは

ログインしていないユーザーが勝手にホーム画面などにアクセスできないようにする仕組みです。
_layout.tsx に書くことで、アプリ起動時に毎回チェックが走ります。

```
useEffect(() => {
  const checkSession = async () => {
    // 現在のセッション（ログイン状態）を確認する
    const { data } = await supabase.auth.getSession();

    if (!data.session) {
      // セッションがない = ログインしていない → ログイン画面へ
      router.replace('/login');
    }
    // セッションがある = ログイン済み → そのまま表示
  }
  checkSession();
}, []);
```

14. useEffect

useEffect とは

コンポーネントが画面に表示されたタイミングで処理を実行するための Hook です。
「画面が開いたときにデータを読み込む」「アプリ起動時にログイン確認する」などに使います。

```

useEffect(() => {
  // ここに実行したい処理を書く
  loadItems();
}, []); // ← 依存配列

// 依存配列の意味
// [] (空)      → 最初の1回だけ実行（コンポーネントが表示されたとき）
// [count]      → count が変わるたびに実行
// なし（省略） → 毎回の再描画時に実行（ほぼ使わない）

```

つまずきポイント①： `useEffect` は `onPress` の中に書けない

```

// 誤り：onPress の中に useEffect を書いてはいけない
onPress={() => {
  useEffect(() => { ... }, []); // エラー！
}}

// 正しい：useEffect はコンポーネントの直下を書く
useEffect(() => {
  loadItems();
}, []);

```

つまずきポイント②： `useEffect` の中に非同期処理を書く方法

`useEffect` のコールバック自体には `async` をつけられません。内部に非同期関数を定義して呼び出します。

```

// 誤り
useEffect(async () => {
  await loadItems();
}, []);

// 正しい
useEffect(() => {
  const load = async () => {
    await loadItems();
  }
  load();
}, []);

```

15. async / await

非同期処理とは

時間がかかる処理（データベースへのアクセス・ネットワーク通信など）のことです。
普通の処理は上から順番に実行されますが、時間のかかる処理を待たずに次の処理が実行されると困ります。

async / await を使うことで、「この処理が終わるまで待ってから次へ進む」と指示できます。

```
// await がないと...
const { data } = supabase.from('expenses').select('*');
console.log(data); // → undefined (まだデータが来ていない)

// await をつけると...
const { data } = await supabase.from('expenses').select('*');
console.log(data); // → データが入っている！
```

await を使う関数には async をつける必要があります。

```
// async をつけた関数の中でだけ await が使える
const loadItems = async () => {
  const { data } = await supabase.from('expenses').select('*');
  setInputItems(data ?? []);
}
```

つまりポイント：コンポーネント自体に async はつけられない

```
// 誤り
export default async function MyScreen() { ... }

// 正しい：コンポーネントの中に async 関数を定義する
export default function MyScreen() {
  const handlePress = async () => {
    await someAsyncFunction();
  }
  return <Button onPress={handlePress} title="実行" />
}
```

16. ユーザー別データ管理

なぜ必要か

`supabase.from('expenses').select('*')` はテーブルの**全データ**を取得します。
複数のユーザーがアプリを使うと、他のユーザーのデータも表示されてしまいます。
ログインしているユーザーのデータだけを取得・保存する必要があります。

`getSession()`：ログインユーザーの情報を取得する

`supabase.auth.getSession()` で今ログインしているユーザーの情報を取得できます。

戻り値の構造はこうなっています：

```
{
  data: {
    session: {
      user: {
        id: "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx", ← ユーザーID
        email: "example@gmail.com"
      }
    }
  }
}
```

// 通常の見方（3行必要）

```
const result = await supabase.auth.getSession();
const session = result.data.session;
const userId = session.user.id;
```

// ネストした分解代入（1行でスッキリ）

```
const { data: { session } } = await supabase.auth.getSession();
const userId = session.user.id;
```

`{ data: { session } }` は**ネストした分解代入**といい、オブジェクトの中のオブジェクトを一度に取り出せる書き方です。

つまずきポイント①：`await` を忘れない

`getSession()` は非同期関数です。 `await` がないと Promise オブジェクト（まだ取得中のもの）が返ってきて、 `session` が使えません。

つまずきポイント②：プロパティにクォートはつけない

`session.user.'id'` は構文エラーです。 `session.user.id` と書きます。

`select()` と `eq()` の違い

よく混同しやすいポイントです。

```
.select('*') // 取得する「列」を指定（絞り込みではない）
.select('id, name') // id と name 列だけ取得
.eq('user_id', session.user.id) // 条件に一致する「行」だけ取得（equal = 等しい）
```

つまずきポイント： `select()` で絞り込もうとしてしまう

`select('user_id')` と書いても「`user_id` 列を取得する」という意味になるだけです。
行の絞り込みには `.eq()` を使います。

データ取得・追加の実装

```
// データ取得：自分のデータだけ取得する
const loadItems = async () => {
  const { data: { session } } = await supabase.auth.getSession();
  const { data } = await supabase
    .from('expenses')
    .select('*')
    .eq('user_id', session.user.id); // 自分のuser_idと一致する行だけ
  if (data) setInputItems(data);
}
```

```
// データ追加：user_id も一緒に保存する
const { data: { session } } = await supabase.auth.getSession();
await supabase.from('expenses').insert({
  user_id: session.user.id, // 誰のデータかを記録する
  name: inputText,
  amount: Number(inputNumber),
  category: category,
  date: '2026-05-10'
});
```

17. RLS (Row Level Security)

RLS とは

行レベルセキュリティの略で、データベース側で「誰がどのデータを操作できるか」を制御する仕組みです。

コードで `.eq('user_id', ...)` と書いて絞り込んでも、APIキーを知っている人がSupabaseに直接アクセスすると、コードを通さずに全データが見えてしまいます。

RLSを設定すると**サーバー側でも**「自分のデータしか見られない」というルールが強制されます。

- コードで絞り込み → アプリからのアクセスを制限
- RLS → 直接アクセスも含めてすべてを制限（本当の意味で安全）

業界の常識：セキュリティは「フロントエンドだけ守っても意味がない」と言われます。コードとサーバーの両方で守ることが大切です。

設定手順

① user_id 列を追加する

Supabase の Table Editor で `expenses` テーブルに列を追加します。

設定項目	値
Name	<code>user_id</code>
Type	<code>uuid</code>
Allow Nullable	ON

Allow Nullable を ON にする理由： 既存のデータには `user_id` が入っていないので、NULL（空）を許可しておく必要があります。

② RLSを有効にする

Supabase ダッシュボード → Authentication → Policies → `expenses` テーブルの **Enable RLS** をクリック。

③ ポリシーを3つ作成する

ポリシー	内容
SELECT	自分のデータだけ取得できる
INSERT	自分のデータだけ追加できる
DELETE	自分のデータだけ削除できる

auth.uid() とは

RLSのポリシーで使うSupabase専用の関数で、「今ログインしているユーザーのUUID」を返します。

```
auth.uid() = user_id
```

SELECTポリシーのUSING式にこれを設定することで、`user_id` が自分のIDと一致する行だけが返るようになります。

全体の仕組み

ユーザーがデータをリクエスト

↓

コードで `.eq('user_id', session.user.id)` を指定 ← アプリ側で絞り込み

↓

RLS が `auth.uid() = user_id` を確認 ← サーバー側でも確認

↓

自分のデータだけが返ってくる

18. プッシュ通知 (expo-notifications)

プッシュ通知とは

アプリを開いていなくても届く通知のことです。

家計簿アプリでは「毎日決まった時間に記録を促す」リマインダーとして使えます。

通知には2種類あります：

種類	内容	特徴
ローカル通知	自分のデバイスで予約して送る	サーバー不要・簡単
プッシュ通知	サーバーから送る	他のデバイスにも送れる

家計簿アプリの毎日リマインドには**ローカル通知**が適しています。

セットアップ

```
npx expo install expo-notifications
```

```
import * as Notifications from 'expo-notifications';
```

通知ハンドラーの設定

アプリのルート（`_layout.tsx`）でコンポーネントの**外**に設定します。

「通知が来たときにどう表示するか」を定義します。これがないと通知が届いても表示されません。

```
// _layout.tsx のコンポーネントの外（上）に書く
Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowBanner: true, // 画面上部にバナーで表示
    shouldShowList: true,   // 通知センター（引き下げ画面）にも残す
    shouldPlaySound: true,  // 通知音を鳴らす
    shouldSetBadge: false,  // アプリアイコンのバッジ数を更新しない
  }),
});
```

deprecated（非推奨）について：

古い書き方の `shouldShowAlert` は現在 deprecated です。

deprecated とは「まだ動くが将来削除されるので新しい書き方に移行してください」という警告です。

Apple・Google が通知の仕様を変更し、「アラート」が「バナー（一時表示）」と「リスト（通知センター）」の2つに分かれたため、`shouldShowBanner` と `shouldShowList` に置き換えられました。

通知の許可を求める

スマホで通知を送るには、まずユーザーに許可してもらう必要があります。
`useEffect` の中で呼び出し、画面表示時に1回だけ許可ダイアログを表示します。

```
useEffect(() => {  
  Notifications.requestPermissionsAsync();  
}, []);
```

通知をスケジュールする

```
await Notifications.scheduleNotificationAsync({  
  content: {  
    title: "家計簿APP", // 通知のタイトル  
    body: "家計簿アプリの記入をお忘れではないですか?", // 通知の本文  
  },  
  trigger: {  
    type: Notifications.SchedulableTriggerInputTypes.DAILY, // 毎日繰り返す  
    hour: 21, // 21時  
    minute: 0, // 0分  
  },  
});
```

つまりポイント（SDK 51以降）：

古い書き方の `trigger: { hour: 21, minute: 0, repeats: true }` は使えなくなりました。
`type: Notifications.SchedulableTriggerInputTypes.DAILY` を使います。

ユーザーが時間を設定できるUIの実装

```
const [hour, setHour] = useState('21');
const [minute, setMinute] = useState('00');

// 時・分の入力欄
<TextInput placeholder='時' value={hour} onChangeText={({text}) => setHour(text)} keyboardType='numeric'
<TextInput placeholder='分' value={minute} onChangeText={({text}) => setMinute(text)} keyboardType='nume

// 設定ボタン
<Button
  title='設定する'
  onPress={async () => {
    try {
      await Notifications.scheduleNotificationAsync({
        content: {
          title: "家計簿APP",
          body: "家計簿アプリの記入をお忘れではないですか?",
        },
        trigger: {
          type: Notifications.SchedulableTriggerInputTypes.DAILY,
          hour: Number(hour),    // 文字列 → 数値に変換
          minute: Number(minute),
        },
      });
      Alert.alert('設定完了', `毎日${hour}時${minute}分に通知します`);
    } catch (e) {
      Alert.alert('エラー', String(e));
    }
  }}
/>
```

つまりポイント：テンプレートリテラルのクォートに注意

'毎日\${hour}時...' はシングルクォートなので \${} が展開されず文字列のままです。
`毎日\${hour}時...` のようにバッククォート (`) を使います。

よく使うパターン集

データ取得 → State に保存

```
const [items, setItems] = useState([]);

const loadItems = async () => {
  const { data: { session } } = await supabase.auth.getSession();
  const { data } = await supabase
    .from('expenses')
    .select('*')
    .eq('user_id', session.user.id);
  if (data) setItems(data);
}

useEffect(() => {
  loadItems();
}, []);
```

Alert（確認ダイアログ）

```
Alert.alert(
  '削除確認', // タイトル
  '本当に削除しますか？', // メッセージ
  [
    { text: 'キャンセル', style: 'cancel' },
    { text: '削除', style: 'destructive', onPress: () => deleteItem() },
  ]
);
```

つまずきやすいポイント一覧

ポイント	注意点
onPress={fn()}	即時実行になる。onPress={() => fn()} と書く
getMonth()	0始まりなので +1 が必要

ポイント	注意点
<code>padStart</code>	文字列メソッドなので <code>String()</code> で変換してから使う
<code>useEffect</code>	<code>onPress</code> の中には書けない
コンポーネントに <code>async</code>	つけられない。内部の関数に <code>async</code> をつける
ファイル名とルートパス	<code>settings.tsx</code> のルートは <code>/settings</code> (sあり)
<code>router.replace</code> vs <code>push</code>	ログイン後は <code>replace</code> 、通常の遷移は <code>push</code>
<code>getSession()</code> に <code>await</code> を忘れる	Promise オブジェクトになり <code>session</code> が使えない
<code>select()</code> で行を絞り込もうとする	<code>select()</code> は列指定、行の絞り込みは <code>.eq()</code>
プロパティにクォートをつける	<code>session.user.'id'</code> ではなく <code>session.user.id</code>
<code>shouldShowAlert</code> (通知)	deprecated。 <code>shouldShowBanner</code> と <code>shouldShowList</code> を使う
通知トリガーの <code>repeats: true</code>	SDK 51以降は <code>SchedulableTriggerInputTypes.DAILY</code> を使う
テンプレートリテラルのクォート	<code>\${}</code> を使うときはシングルクォートではなくバッククォート